



CONVERTING E-R DIAGRAMS TO RELATIONAL MODEL

SHENKAR

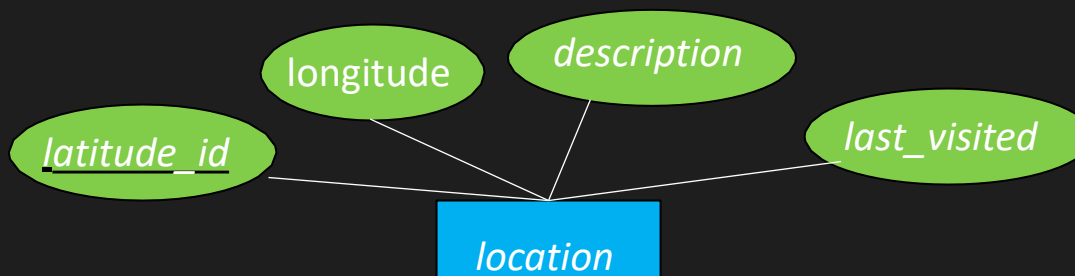
ERD → RELATIONAL MODEL

- המטרה: מעבר מדיאגרמת ER לסכמה שממנה ניתן לבנות בסיס נתונים.
- המעבר מ ERD למודל רלציוני וממנו לקוד SQL הוא יחסית פשוט:
 - ישנה חפיפה בין מודל ה ER ל מודל הרלציוני.
 - ההבדל העיקרי הוא בין המאפיינים המורכבים/ מרובים ב ER לעומת מאפיינים חד ערכים במודל הרלציוני
- תהליך ההמרה כולל שלושה חלקים:
 - הגדרת סכמה
 - הגדרת PK
 - הגדרת FK

מעבר של ENTITY

- בהנתן יישות עם מאפיינים $a_1, a_2, \dots, a_n$
(נניח שהמאפיינים הם פשוטים)
- ניצור טבלה עם אותו שם ועם אותם מאפיינים
- המפתח יהיה אותו מפתח

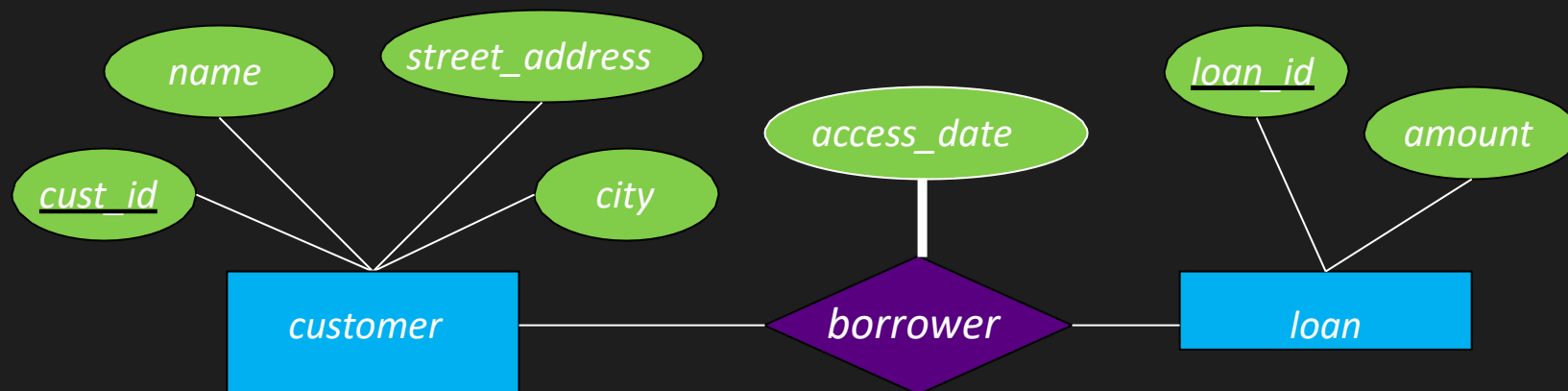
- Entity: location in E-R diagram:



ההמרה לסכמה רלציונית תהיה:

```
location(latitude_id, longitude, description,  
last_visited)
```

- E-R diagram for customers and loans:



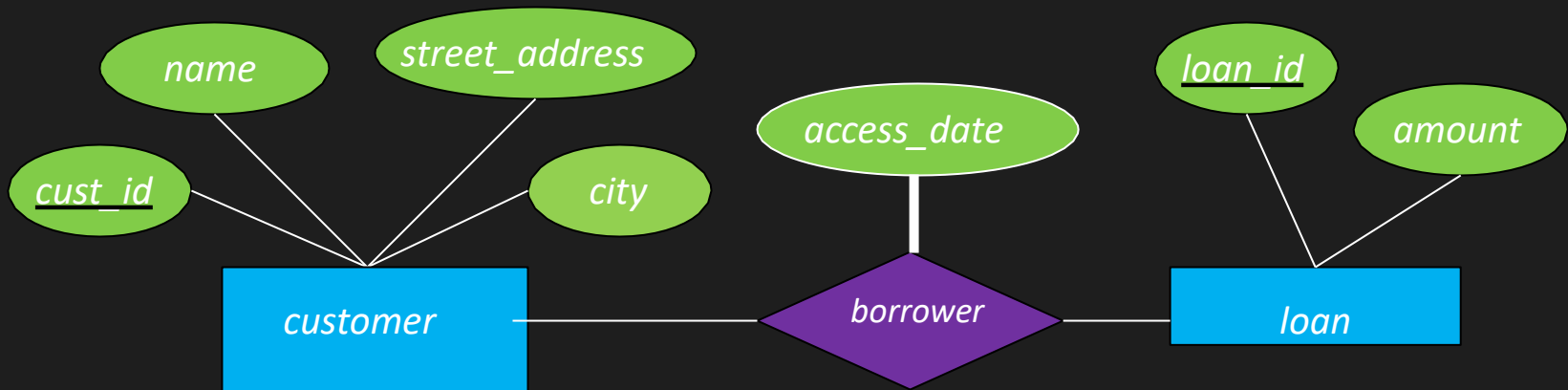
• ההמרה לסכמה רלציונית תהיה:

- `customer(cust_id, name, street_address, city)`
`loan(loan_id, amount)`

RELATIONSHIP - PRIMARY KEYS

- בחיבור של 2 יישויות :
 - במקרה של many-to-many - המפתחות של הישויות הופכים להיות המפתחות של טבלת הקשר
 - במקרה של one-to-one – כל אחד ממפתחות הישויות יכול להיות מועמד למפתח הקשר.
 - במקרה של many-to-one or one-to-many - המפתח בצד של ה many - הוא המפתח של הקשר

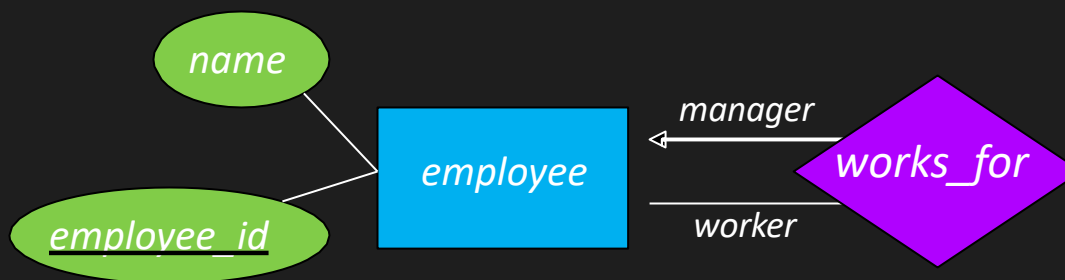
דוגמה #1



- הסכמה עבור borrower תהיה:
- המפתח מ customer – cust_id
- המפתח מ loan – loan_id
- מאפיין access_date
- הקשר הוא many-to-many

`borrower(cust_id, loan_id, access_date)`

דוגמה 2#



- הסכמה עבור employee :

`employee(employee_id, name)`

- הסכמה עבור works_for :

○ יש קשר אחד לרבים בין המנהל לעובד

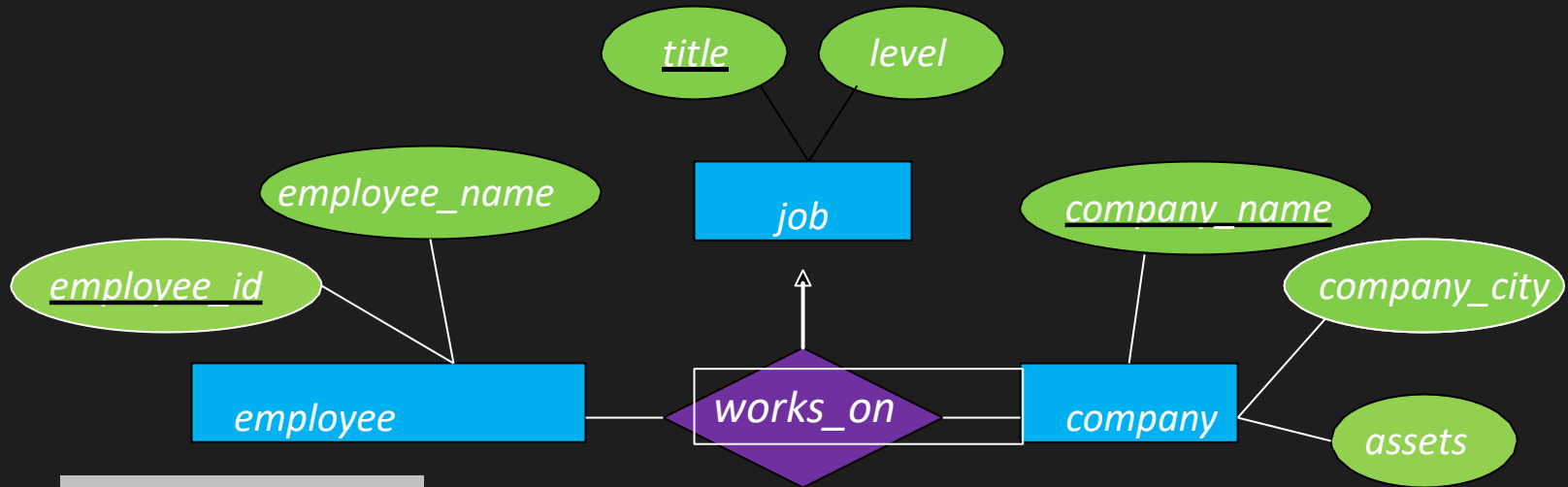
○ הצד של הרבים (employee_id) יהיה המפתח.

`works_for(employee_id, manager_id)`

N-ARY RELATIONSHIP PRIMARY KEYS

- עבור קשר של יותר מ 2 יישויות:
- במקרה של **many-to-many** – המפתח הוא חיבור של כל המפתחות מהישויות המשתתפות בקשר
- במקרה של **one-to-many** – המפתח הוא חיבור של כל המפתחות מהישויות המשתתפות בקשר ללא הישות של ה **one**

N-ARY RELATIONSHIP דוגמה



job(title, level)

employee(employee_id, employee_name)

company(company_name, company_city, assets)

works_on(employee_id, company_name, title)

SCHEMA COMBINATION

- לא תמיד צריך ליצור טבלה נפרדת על מנת לייצג קשר.
- ניתן לייצג את הקשר ע"י הוספת המאפיין לטבלה העיקרית
- יתרונות:
 - חוסך FK
 - חוסך JOIN
- חסרונות:
 - הרבה ערכים עם NULL
 - קושי בשינוי סוג הקשר בעתיד (לדוגמה: מיחיד ליחיד לרבים)

דוגמה #1



- הקשר בין מנהל לעובד הוא יחיד לרבים
- את הסכמה הבאה:

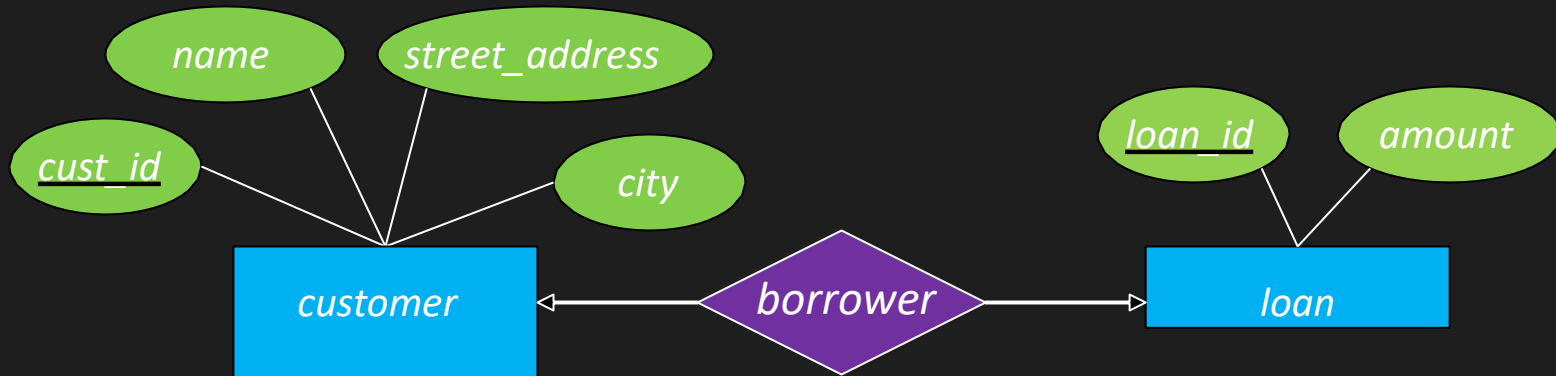
- `employee(employee_id, name)`
`works_for(employee_id, manager_id)`

- ניתן להפוך

`employee(employee_id, name, manager_id)`

עבור עובד ללא מנהל – `manager_id` יהיה null

דוגמה 2#



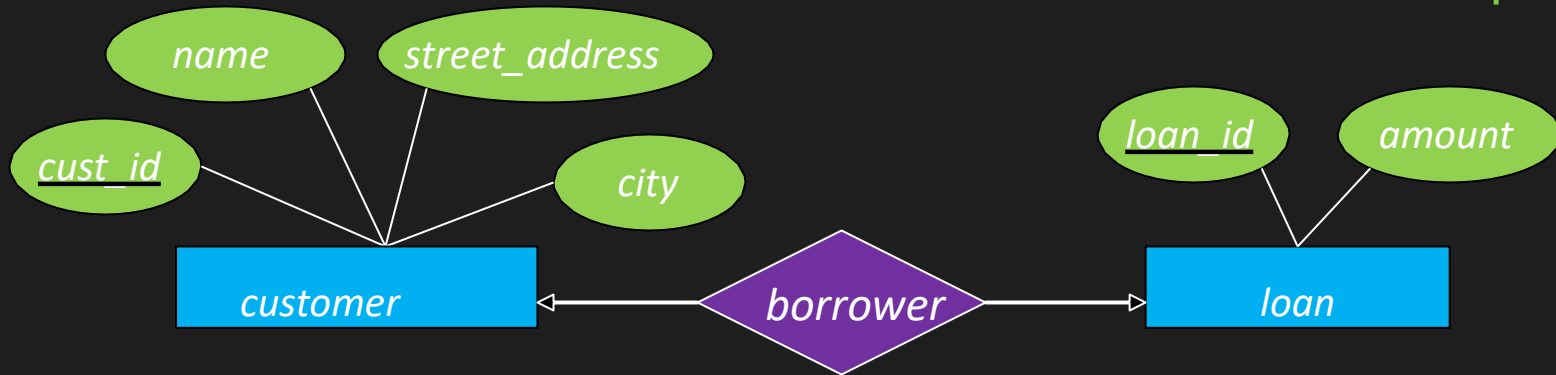
• ייצוג של One-to-one:

- `customer(cust_id, name, street_address, city)`
`loan(loan_id, amount)`
`borrower(cust_id, loan_id)`

• ניתן לממש את הקשר גם באמצעות שיוך הקשר (`borrower`) ל
`customer` או `loan`

האם זה משנה?

המשך דוגמה 2#



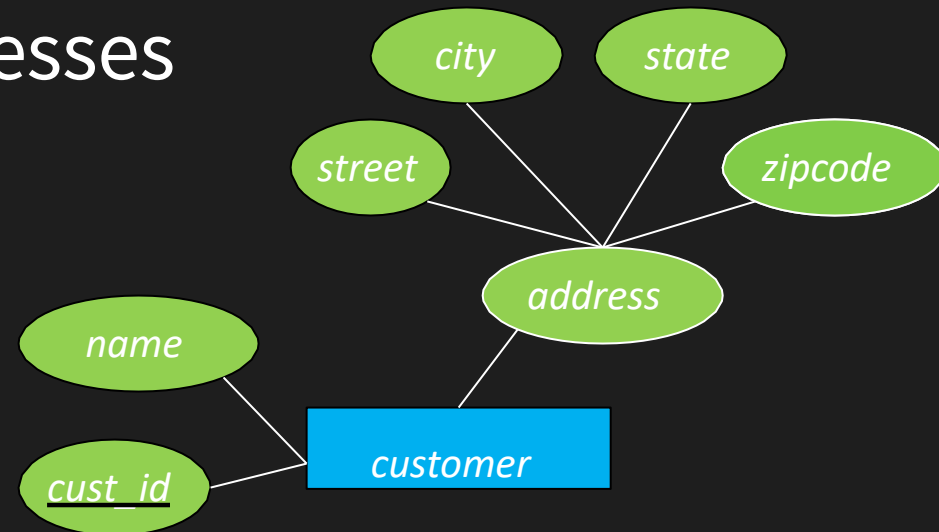
- אם נשייך את הקשר לטבלת **customer**:
customer(cust_id, name, street_address, city, loan_id)
loan(loan_id, amount)
- יהיו לנו NULL עבור לקוחות ללא הלוואה
- לכן עדיף:
customer(cust_id, name, street_address, city)
loan(loan_id, cust_id, amount)
– אין NULL מיותרים

COMPOSITE ATTRIBUTES

- המודל הרלציוני לא תומך במאפיינים מורכבים
- במעבר של מאפיינים מורכבים ממודל ה-ER למודל הרלציוני – נפריד כל מאפיין לעמודה

דוגמה

- Customers with addresses



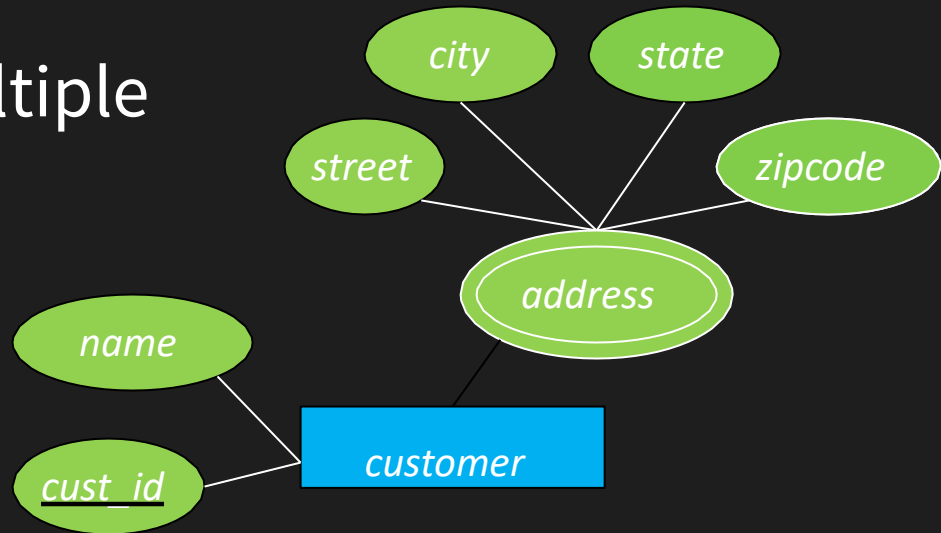
כל חלק של הכתובת הופך להיות מאפיין:

`customer(cust_id, name, street, city, state, zipcode)`

MULTIVALUED ATTRIBUTES

- עבור מאפיין בעל ערכים מרובים נצטרך טבלה נפרדת
- המפתח עבור הטבלה הזו יהיה המפתח של היישות העיקרית.

- Customers with multiple addresses



- Create separate relation to store each address

`customer(cust_id, name)`

`cust_addrs(cust_id, street, city, state, zipcode)`

–מפתח מרובה שדות הוא לא אדיאלי – יקר מבחינת ביצועים